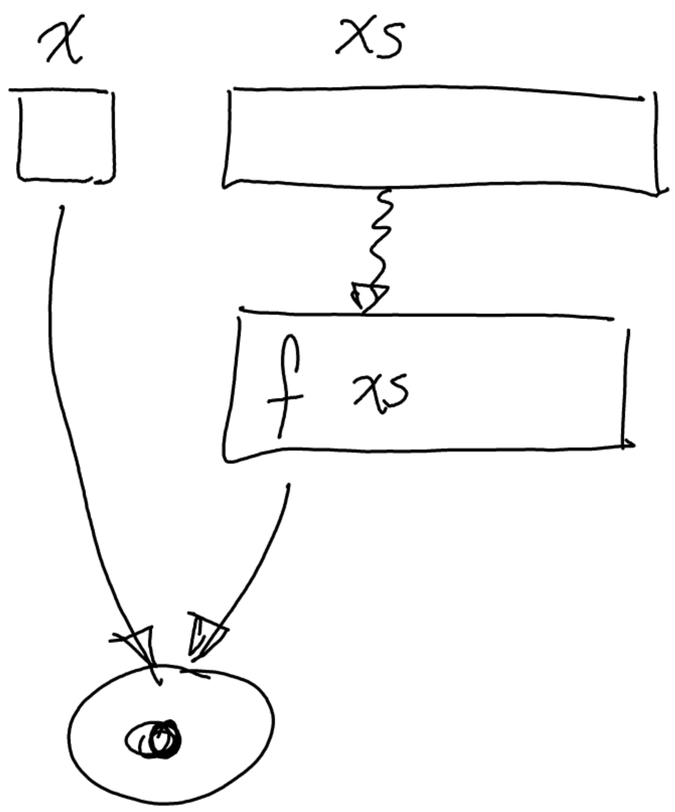
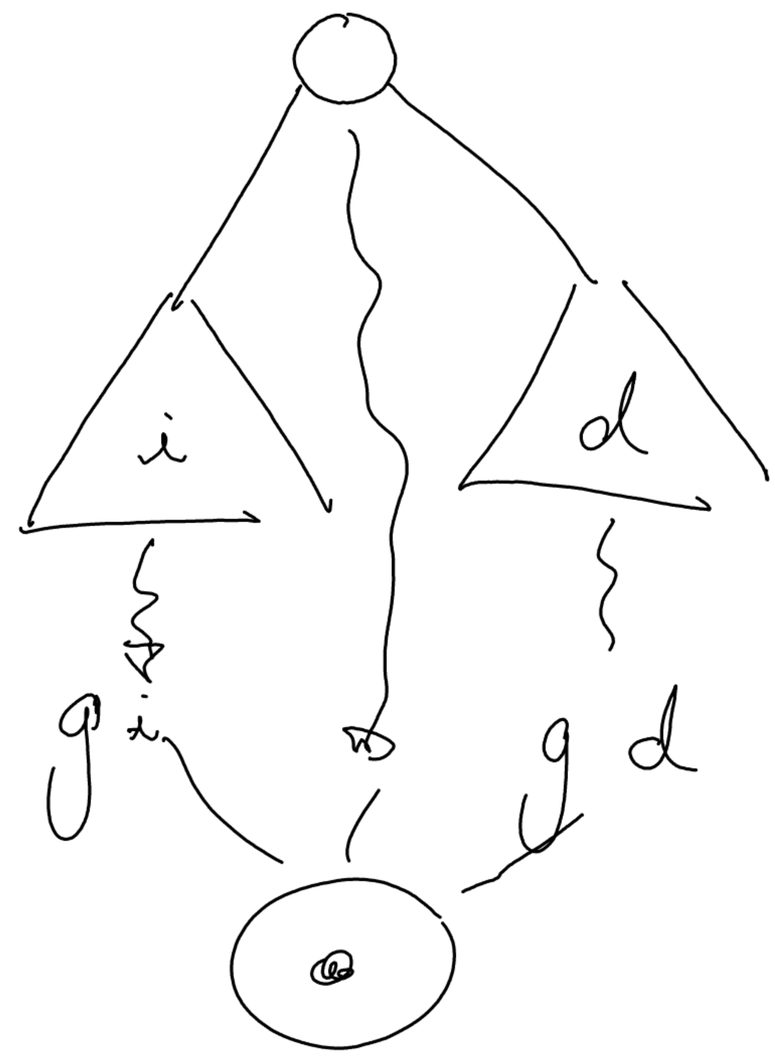
```
data [a] = [] | a : [a]
           Nil | Cons a [a]
```





cantNodos :: AEB a → Int

cantNodos t = foldAEB

(\ a → 1)

(\ri a rd →

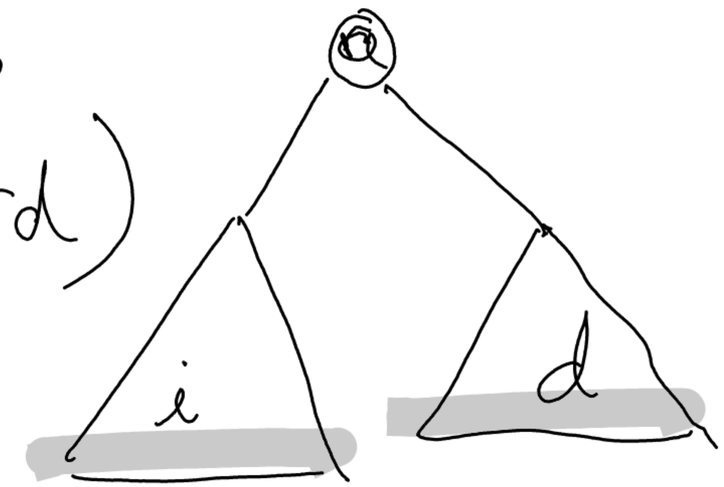
1 + ri + rd)

(const 1)

t

se pueden sacar.

cantNodos (Hoja 2) ⇝ 1

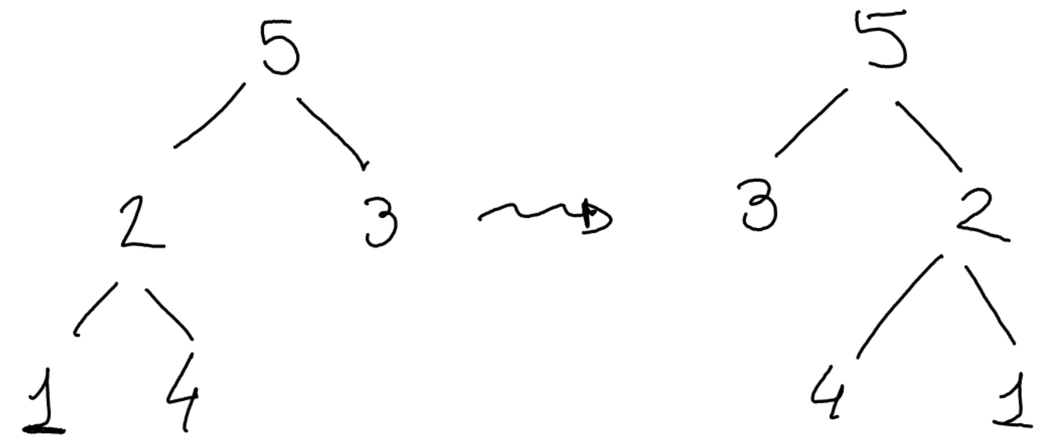


1 + cantNodos i + cantNodos d

cantNodos = foldAEB (const 1) (\i _ d → 1 + i + d)

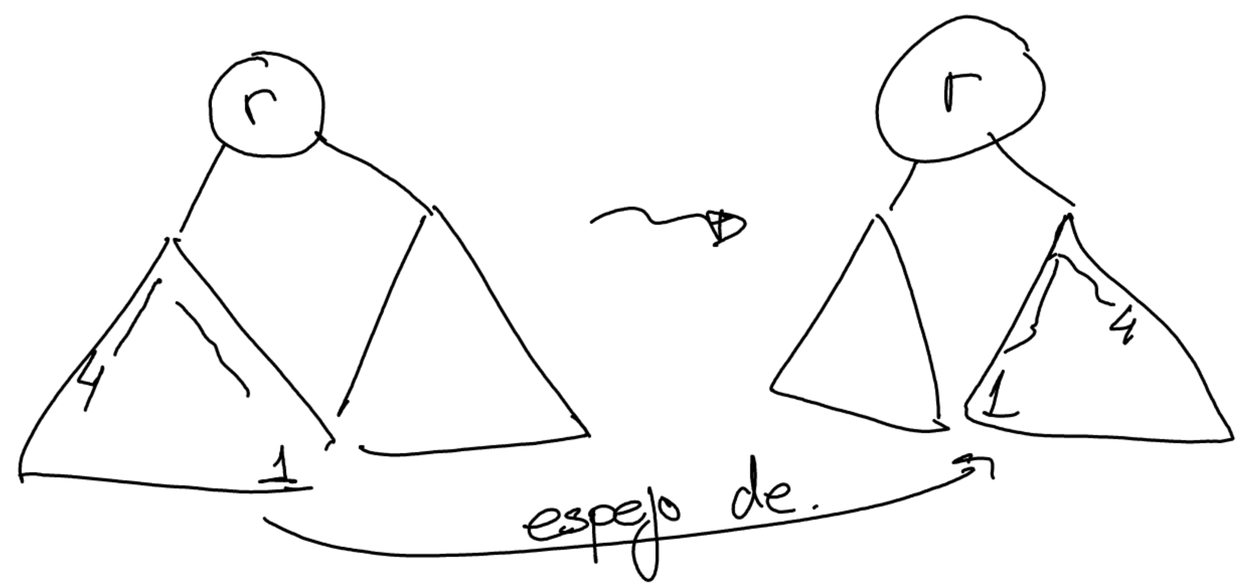
```
espejo :: AEB a -> AEB a
espejo = foldAEB Hoja
    (\i r d ->
        Bin d r i)
```

! ya es el espejo del árbol izquierdo original.




```
Hoja :: a -> AEB a
```

$$\underbrace{\cancel{\lambda n} \to \text{Hoja}\ \cancel{n}}_{a \to \text{AEB } a}$$



```
data AB a = Nil | Bin (AB a) a (AB a)

foldAB :: b -> (b -> a -> b -> b) -> AB a -> b

recrAB :: b -> (AB a -> b -> a -> AB a -> b -> b) -> AB a -> b

recrAB fNil fBin t =
    case t of
        Nil -> fNil
        Bin i r d -> fBin i (rec i) r d (rec d)
    where
        rec = recrAB fNil fBin
```

```
data Poli a = X | Cte a | Soma (Poli a) (Poli a) | Prod (Poli a) (Poli a)

evaluar :: (Num a) => a -> Poli a -> a
evaluar x X = x
evaluar x (Cte n) = n
evaluar x (Soma p q) = (evaluar x p) + (evaluar x q)
evaluar x (Prod p q) = (evaluar x p) * (evaluar x q)
```

$X^2 + 2X + 1 \equiv$ Soma (Soma (Prod X X) (Prod (Cte 2) X)) (Cte 1)

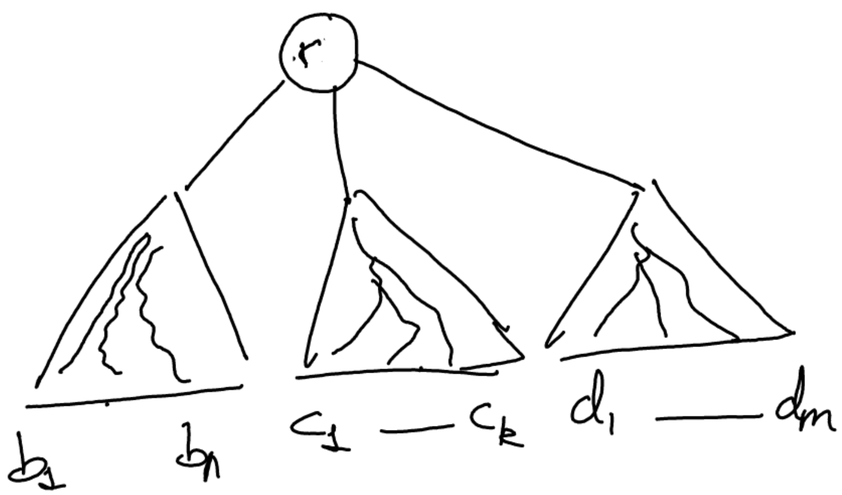
```
foldPoli :: b -> (a -> b) -> (b -> b -> b) -> (b -> b -> b) -> Poli a -> b

evaluar' x = foldPoli x id (+) (*)
```

```
data RoseTree a = Rose a [RoseTree a]

foldRT :: (a -> [b] -> b) -> RoseTree a -> b
foldRT f (Rose r ts = f r (map (foldRT f) ts))

ramas :: RoseTree a -> [[a]]
ramas = foldRT (\r res ->
    if null res then [[r]]
    else
        map (r:) (concat res)
)
```




```
(++) :: [c] -> [c] -> [c]
concat :: [[c]] -> [c]
concat res ~> [b1, b2 ... bn, c1 ... ck, d1 .. dm]
map :: (a -> b) -> [a] -> [b]
```

```
r : b1
  ⋮
r : bn
r : c1
  ⋮
r : ck
r : d1
  ⋮
r : dm
```

```
res [
    [b1, b2 .. bn]
    [c1, c2 ... ck]
    [d1, ... dm]
]
```

b1 :: [a]

[[a]]

res :: [[[a]]]